MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Department of Instrumentation and Control Engineering

VIRTUAL INSTRUMENTATION

LABORATORY MANUAL- ICE 3243

VI SEM. B.Tech.

Name of the student:……

Registration Number:……...

January 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CERTIFICATE

This is to certify that the Laboratory Manual/Journal for the lab titled **VIRTUAL INSTRUMENTATION (ICE 3243)** submitted by Mr./Ms._____ (Reg. No :_____) of sixth semester, Electronics and Instrumentation Engineering for the academic year _____ , as per laboratory course requirements, which has been evaluated and duly certified.

Place: Manipal
Date:

Lab In-Charge

CONTENTS

Sl. No.	Experiment Name	Date of experimentation	Page No.
0	Introduction to LabVIEW		1
1	Arithmetic and logical operations		4
2	Modular programming		6
3	Loops and shift registers		8
4	Feedback node and communicating among loops		9
5	Array operations		11
6	Clusters and graphical display		12
7	String handling and file I/O		13
8	Programming structures		14
9	IMAQ and VISION tool box		15
10	Control and simulation toolbox.		16

Evaluation Plan:

Continuous Evaluation : **60%** (Preparation, Lab performance, Journal, Assignments and Regularity)

End Semester Lab Test : **40%**

EXPERIMENT NO.0	Date:

Introduction to LabVIEW

Aim: To understand the principles of Virtual Instrumentation (VI).

Theory:

Virtual Instrumentation: Virtual instrumentation is the use of customizable software and modular measurement hardware to create user-defined measurement systems, called virtual instruments. Traditional hardware instrumentation systems are made up of pre-defined hardware components, such as digital multimeters and oscilloscopes that are completely specific to their stimulus, analysis, or measurement function. Because of their hard-coded function, these systems are more limited in their versatility than virtual instrumentation systems. The primary difference between hardware instrumentation and virtual instrumentation is that software is used to replace a large amount of hardware. The software enables complex and expensive hardware to be replaced by already purchased computer hardware; e. g. analog-to-digital converter can act as a hardware complement of a virtual oscilloscope, a potentiostat enables frequency response acquisition and analysis in electrochemical impedance spectroscopy with virtual instrumentation.

LabVIEW: Laboratory Virtual Instrumentation Engineering Workbench (LabVIEW) is a graphical programming language that uses icons instead of lines of text to create applications. In contrast to text-based programming languages, where instructions determine program execution, LabVIEW uses dataflow programming, where the flow of data determines execution. In LabVIEW, a user interface can be built by using a set of tools and objects. The user interface is known as the front panel. Then code can be added using graphical representations of functions to control the front panel objects. The block diagram contains this code. In some ways, the block diagram resembles a flowchart.

LabVIEW programs are one of the suitable for virtual instruments, or VIs, because their appearance and operation imitate physical instruments, such as oscilloscopes and multimeters. Every VI uses functions that manipulate input from the user interface or other sources and display that information or move it to other files or other computers.

A VI contains the following three components:

Front panel - Serves as the user interface.

Block diagram - Contains the graphical source code that defines the functionality of the VI.

Icon and connector panel - Identifies the VI so that the VI can be used in another VI. A VI within another VI is called a subVI. A subVI corresponds to a subroutine in text-based programming languages.

The front panel is the user interface of the VI. The front panel is built with controls and indicators, which are the interactive input and output terminals of the VI, respectively. Controls are knobs, pushbuttons, dials, and other input devices. Indicators are graphs, LEDs, and other displays. Controls simulate instrument input devices and supply data to the block diagram of the VI. Indicators simulate instrument output devices and display data the block diagram acquires or generates. After the front panel is build, add code using graphical representations of functions to control the front panel objects. The block diagram contains this graphical source code. Front panel objects appear as terminals on the block diagram.

Additionally, the block diagram contains functions and structures from built-in LabVIEW VI libraries. Wires connect each of the nodes on the block diagram, including control and indicator terminals, functions, and structures.

LabVIEW Palettes

LabVIEW palettes give provide the options needed to create and edit the front panel and block diagram. The Tools palette is available on the front panel and the block diagram. A tool is a special operating mode of the mouse cursor. By selecting a tool, the cursor icon changes to the tool icon. Use the tools to operate and modify front panel and block diagram objects.

Select Window » Show Tools Palette to display the Tools palette.

The Tools palette can be placed anywhere on the screen. If automatic tool selection is enabled and as the cursor is moved over objects on the front panel or block diagram, LabVIEW automatically selects the corresponding tool from the Tools palette.

The Controls palette

The Controls palette is available only on the front panel. The Controls palette contains the controls and indicators used to create the front panel. to display the Controls palette, Select Window » Show Controls Palette or right-click the front panel workspace. The Controls palette can be placed anywhere on the screen.

The Functions palette

The Functions palette is available only on the block diagram. The Functions palette contains the VIs and functions used to build the block diagram. To display the Functions palette, select Window » Show Functions Palette or right-click the block diagram workspace. The Functions palette can be placed anywhere on the screen.

Dataflow Programming

LabVIEW follows a dataflow model for running VIs. A block diagram node executes when all its inputs are available. When a node completes execution, it supplies data to its output terminals and passes the output data to the next node in the dataflow path.

Procedure:

1. Open NI LabVIEW and press <Ctrl_N> to open a blank VI.
2. Press <Ctrl_T> to tile front panel and block diagram windows.

Exercise:

1. Design a front panel containing a **slider** and a **numeric indicator** that updates continuously.

EXPERIMENT NO.1	Date:

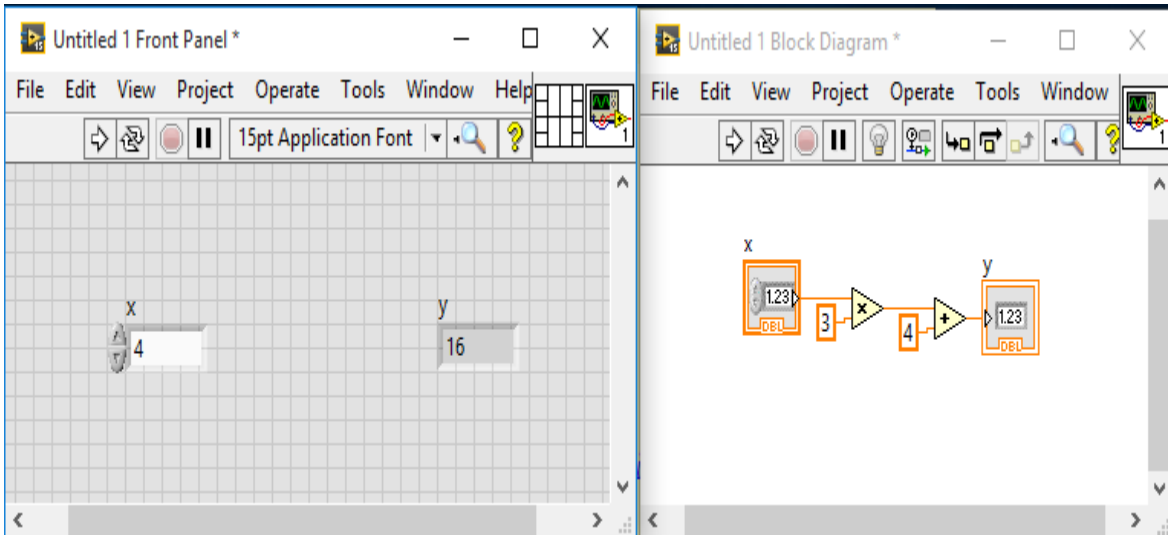
Arithmetic and logical operations

Aim: To perform basic arithmetic and logical operations using LabVIEW.

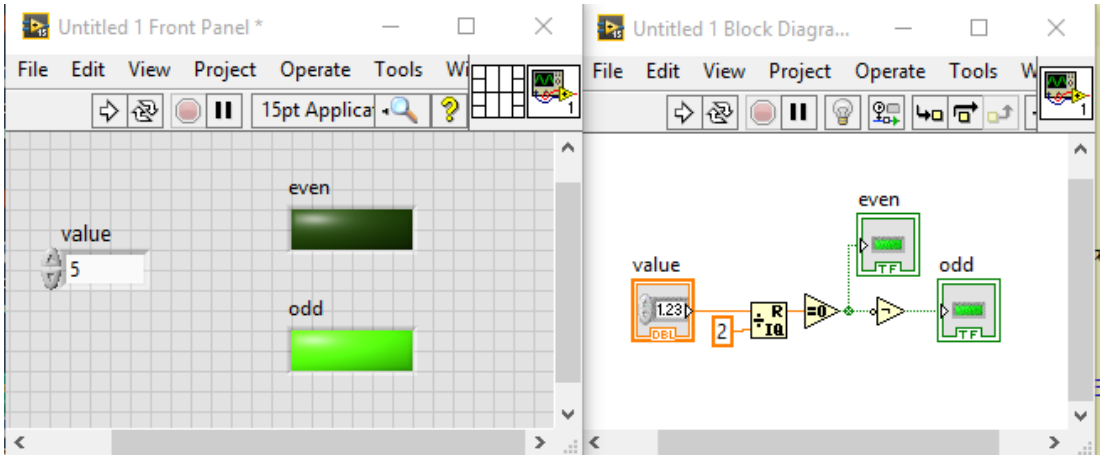
Instructions:

Step 1: Select controls by right clicking on the front panel and choose the specific type of control similarly choose indicators.
Step 2: Different operators such as addition, subtraction, multiplication, less than, greater than, Boolean, etc can be performed by right clicking on block diagram.
Step 3: Using wires inputs and outputs are connected to the respective operators in the block diagram panel.
Step 4: Enter the input values in front panel.
Step 5: Observe the output by performing single step and continuous mode execution.

Example 1: Execute a VI to perform the operation of y=3x+4.



Example 2: Execute a VI to perform the operation of checking a given number is odd or even



Exercise:

1. Develop a VI to convert **minutes into hours and seconds**.
2. Create a VI to perform $y = \sqrt{2 * b + a^3} + log_{10}(ab)$ using functions and formula node.
3. Implement the Boolean logic $Y = \overline{\bar{A} + (B \oplus C)}.(D + \overline{B})$ and also verify the truth table.
4. Build a VI to calculate parking charges based on duration of parking with separate time-based tariff structures for two-wheelers and four-wheelers. Use a toggle switch to select the vehicle type.
5. Create a VI find the magnitude and phase angle of a complex number.

EXPERIMENT NO.2	Date:

Modular programming

Aim:

To perform modular programming in LabVIEW.

Instructions:

A. Creating SubVI

Step 1: Build a VI with front panel and block diagram
Step 2: Assign terminals to control and indicators using control pane.
Step 3: Create icon
Step 4: Save the file with extension .vi
Step 5: Call the file in the main program using 'select vi' option in block diagram pane.
Step 6: If multiple SubVI are to be used group can be saved as .llb (LabVIEW library)

B. Displaying subVI or express VI as icon or expandable node:

C. Creating EXE from a VI

Step 1: Right click 'build specifications' in the 'project explorer' window and select 'New'>> 'Application (EXE)' from the shortcut menu
Step 2: Modify the file name of the target and destination directory for the application in the 'Application Information category'.
Step 3: Select the 'Application Information category'.
Step 4: Change the target filename to your 'filename .exe'.

Exercise:

1. Create a subVI to compute $(x_2 - x_1)^2$ and use this in the main VI to compute distance between two points $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.
2. Create a VI to find the decimal equivalent of a binary number using subVI.
3. Create a VI to compute full adder logic using half adder logic as subVI.
4. Create a VI to calculate compound interest using simple interest subVI.

EXPERIMENT NO.3	Date:

Loops and shift registers

Aim:

To perform the operation using loops in LabVIEW.

Instructions:

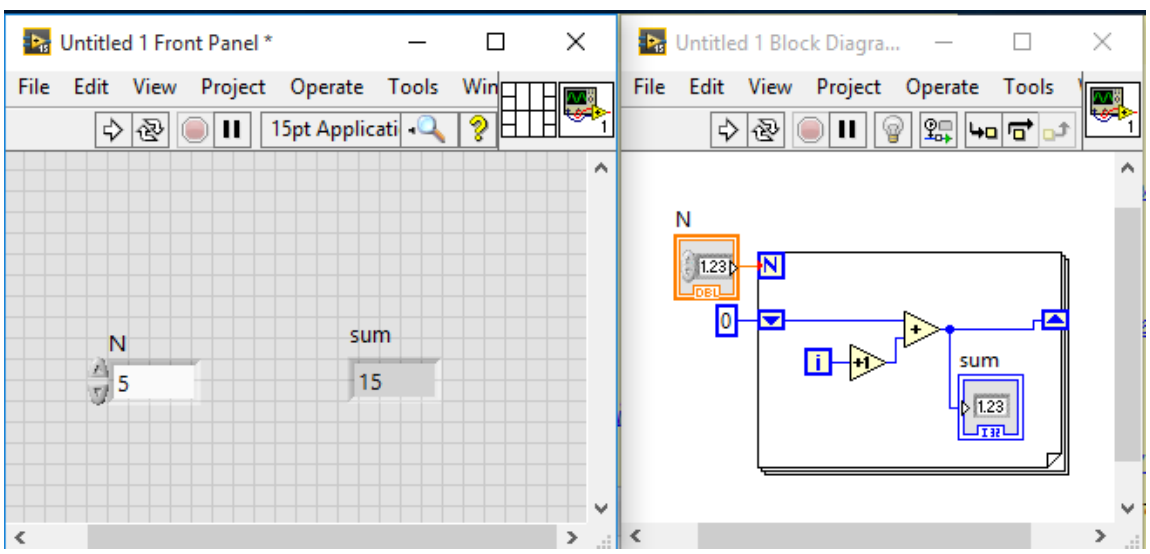
A. **While loop:** to be used when condition needs to be checked after an iteration.

Step 1: In block diagram panel, right click and select 'Programming'>> 'structures' >> 'while loop'.
Step 2: choose the terminating condition.

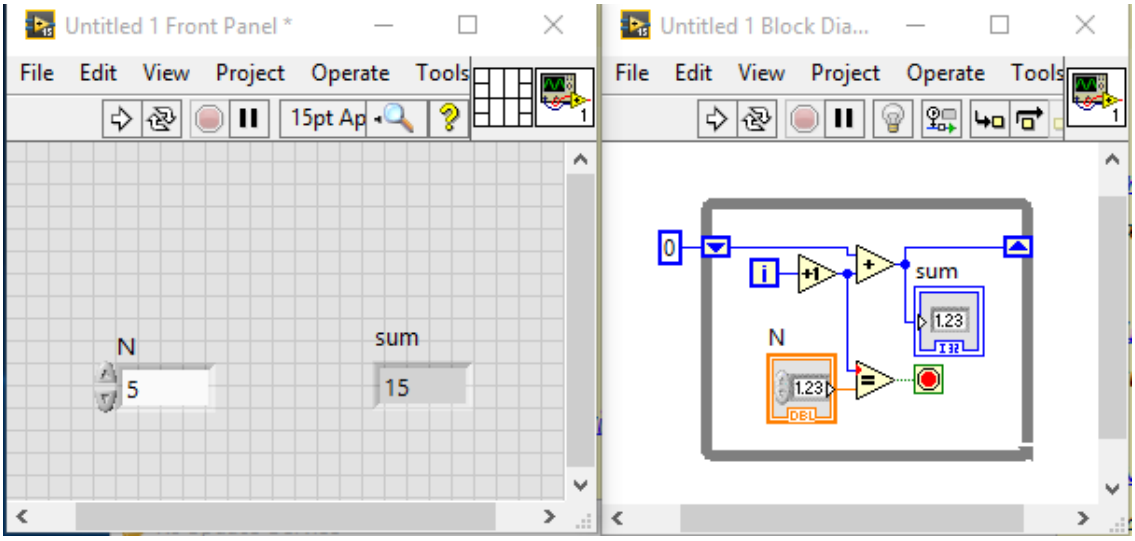
B. **For loop:** to be used when condition needs to be checked before every iteration.

Step 1: In block diagram panel, right click and select 'Programming' >> 'Structures' >> 'for loop'.
Step 2: Assign the number 'N', which means number of times loops should be execute.

Example 1: Compute the sum of first N numbers.



Using 'For loop'



Using 'while loop'

Exercise:

1. Find the sum of first 'N' odd numbers
2. Create a VI to animate a horse running. Use the picture ring control to insert and display pictures to be animated.
3. Create a program for finding the factorial of a number and use this as subVI to compute ${}^{N}P_r$ and ${}^{N}C_r$.
4. Compute the sum of digits of a given integer using for loop.
5. Find the sum of values lying between 'P' and 'Q'. Where 'P' and 'Q' are positive integers and P>Q.

Homework:

1. Find the reverse of a number using a While Loop (without using reverse function).
2. Count the number of positive and negative values in a given data set.

EXPERIMENT NO.4	Date:

Feedback node and communicating among loops

Aim:

To perform operations by using the feedback node of LabVIEW and communicate between multiple loops.

Instructions:

A: Feedback Node - Feedback node stores data when the loop completes an iteration, sends that value to the next iteration of the loop and transfers any data type.

Step 1: Select 'Feedback Node' on the structures palette and place it in for or while loop.

Step 2: Initialize a feedback node by right clicking the feedback node and select 'initializer terminal' from the short cut menu. Add wire from outside the loop to the 'initializer terminal'.

B: Control timings- LabVIEW allows to add wait function in the loop, to wait an amount of time in millisecond before the loop executes the next iteration.
To select wait function right click on block diagram, select 'Programming' >> 'timing'.

C: Communicating among multiple loops
Local variables transfer data within a single vi.
Right click on an object's terminal and select 'create'>> 'local variable'.

Global variables are built-in LabVIEW objects, which are used to access and pass data among several vi's that run simultaneously.
Right click on block diagram, select 'structure' >> 'global variable', in pop-up window select the variable.

Exercise:

1. Design a VI to display the **previous and current values** of a continuously changing signal using feedback node.
2. Create a VI where two loops run in parallel and **share a threshold value**.
3. Create a VI which converts a decimal number to a binary number (displayed in numeric indicator).
4. Create a VI which will increment a counter every time up button is pressed on decrements the counter value every time the down button is pressed. A LED glows if counter is equal to zero.

EXPERIMENT NO.5	Date:

Arrays

Aim:

To perform the array operations in LabVIEW

Instructions:

Creating a one-dimensional array control/indicators/constant
Step 1: In front panel right click >>'Controls' >> 'Modern'>>'Array, Matrix & Clusters palette'.
Step 2: Insert an object in the array. The array data type will be assigned as that of the object inserted.
Step 3: To create multidimensional array, Right click on array and select 'Add Dimension'.
Step 4: From the array particular set of elements can be made visible by varying the values of 'index' provided next to array.

Exercise:

1. Display 10 random integers between 2 and 50.
2. Count the number of elements greater than a given value in an array.
3. Find the sum of both diagonals of a square matrix without built in functions.
4. Write a program to arrange a given set of number in ascending and descending order.
5. Develop a VI that counts how many numbers in an array are **even** and how many are **odd**.

Homework:

1. Perform A×B where 'A' and 'B' are 3X3 matrices.
2. Display a square matrix of 5 rows and columns elements, such that addition of individual row elements should be same and so the column elements.

EXPERIMENT NO.6	Date:

Clusters and graph & chart

Aim: To perform the operation on clusters and graphs in LabVIEW.

Clusters are group of data with element of mixed type.

Instructions:

To create clusters, select cluster by right clicking in front panel and select 'controls'>>'All controls'>>'Arrays & cluster palette', place it on front panel. Drag and drop objects or elements into the cluster.

Plotting Data:

1. Graph: display typical data
2. Chart : displays data at continuous rate
    a. Strip chart: shows continuous data and continuously scroll from left to the right
    b. Scope chart: shows one item, and scroll partway across left to right.
    c. Sweep chart: shows old data on right and new data on left.
3. XY graph: two dimension plot
4. 3D plot
5. Intensity graph:

Right click on front panel, select 'graphs and charts'.

Exercise:

1. Build a VI that displays two plots, a random plot and a running average of the last four points, on a waveform chart in sweep update mode.
2. Build a VI which acts as function generator.
3. Plot the equations $y1=3x^2+2x-1$ and $y2 = x-3$, in steps of 0.1 from range 0 to 5. Also check if the y1 and y2 meets, if they meet display its coordinates.
4. Build an array of cluster controls in which each cluster consists of a numeric control, string and two 1D numeric array (with six elements). This forms a database of marks and attendance of students. Numeric control indicates the roll number, first array indicates the attendance in percentage, the second array indicates the test marks in six subjects and the string indicates the name. Input the roll number, subject in which mark or attendance is to be changed and the new mark or attendance. Display the changed database on a separate array indicator.

Homework:

1. Create a cluster-based database for **sensor ID, sensor type, and last three readings**.

EXPERIMENT NO.7	Date:

Strings and File I/O

Aim:

To perform the operations on strings and file I/O in LabVIEW.

String is a sequence of displayable or non-displayable ASCII characters.
String control and indicator is located on the 'Controls'>> 'Text controls'>>'Text indicators palettes' in the front panel.

File I/O records or reads data in a file. Functions related to file I/O is located by right clicking the block diagram 'Functions'>>'All functions'>> 'File I/O'.

Exercise:

1. Find the length of the given string A, if its length is less than 'N' concatenate string A with string B. Then find the final length.
2. Read the string from file xx.txt convert the text in the file to uppercase and store the resultant in yy.txt.
3. Count the number of spaces in a given string.
4. Count the number of characters in a text file.
5. Replace all occurrences of a given character with another character in a string.

EXPERIMENT NO.8	Date:

Structures

Aim:

To perform the operation on structures in LabVIEW.

Structures are used in block diagram to repeat blocks of code and to execute code conditionally or in a specific order. Section of block diagram inside the structure border is called 'subdiagram'. The terminals that feed data into and out of structures are called 'tunnels'.

Following structures can be used in LabVIEW

1. Loop's
2. Case structure
3. Sequence structure
4. Event structure
5. Timed structure
6. Diagram disable structure
7. Conditional disable structure

To add structures right click on block diagram, select 'programming'>> 'structures'.

Exercise:

1. Create a program to perform the following, a. y= A+B+C if 0<N<5, b. $y=A^2+2B-C$ if 6<N<20, else y=0.
2. Implement a grading system using Case Structure based on marks.
3. In a given set 'N' consisting of 20 numbers, create a set 'M' which contains only numbers greater than a given threshold.
4. Execute the given equation $y = \{(A^2-B^2) \times (B+C) \times (C-A)^2\}-2$ using stacked sequence and flat sequence.
5. Select and execute different mathematical operations using Event Structure.
6. Execute a program to compute y = Asinx + Bcosx (log(1/p-1)). If the resultant is greater than 5 the program should halt and display error message.

EXPERIMENT NO.9	Date:

IMAQ and VISION tool box

Aim:

To perform the vision related operations in LabVIEW.

Instructions:

For Vision toolbox right click on block diagram select 'Vision and motion'

Exercise:

1. Find the Histogram and Histograph of an Image
2. Write a program to vary the brightness and contrast of an image.
3. For an image compare the effect of Smoothing, Gaussian, Gradient, and Laplacian filters.
4. Find the FFT spectrum of an image.
5. Apply the morphological operation like threshold and compare the results.

EXPERIMENT NO.10	Date:

Control and simulation toolbox.

Aim:

To perform the control applications using LabVIEW.

Instructions:

For control system toolbox right click on block diagram, select 'Control and Simulation tool'.

Exercise:

1. Given a numerator and denominator array represent it in TF form, and then convert the same to state space form.
2. Compute the time response analysis of the given second order system, compare the result by using 'LabVIEW Mathscript'.
3. Comment on stability using root locus and bode for a given system, compare the result by using 'LabVIEW Mathscript'.
4. Identify the TF model for a given RC circuit using system identification tool.